

The possibility of Options 1 and 2 are remote, compelling us to choose between Options 3 or 4. As a solution like that suggested in Option 3, a project named BTRFS, guided by Chris Mason at Oracle is under development, having been merged to a "rc" pre-release of the current Linux kernel (2.6.30), and is under testing. Definitely, this is going to take a long time as ZFS itself was under development for five years. Solution 4, which is through a utility called FUSE and seems the most stable option as of now, is what I am going to discuss as we go on.

FUSE

Filesystem in Userspace, or FUSE, helps implement a fully-functional filesystem in a userspace program rather than directly in the kernel. It is implemented in OSs like Linux, FreeBSD, etc. Its components (as of version 2.7.4) consist of a FUSE kernel module, a FUSE library containing *libfuse* and *liblockmgr*, and a special file descriptor like a device file in Linux named */dev/fuse*, used for communication between the kernel module and the userspace library. For user convenience, a program named 'fusemount' is provided along with the FUSE package as an easy usermode tool to link up between the user-defined filesystem and the FUSE module.

ZFS on FUSE

ZFS on FUSE is a project under development by Ricardo Manuel da Silva Correia, a computer engineering student, and is sponsored by Google as part of Google Summer of Code 2006. So after completion of this project, ZFS will have a port on the FUSE framework, which effectively will mean operating systems like Linux can use ZFS. A rough performance comparison of ZFS on FUSE with NTFS-3G, XFS and JFS can be found at www.csamuel.org/2007/04/25/comparing-ntfs-3g-to-zfs-fuse-for-fuse-performance.

How it works

The *zfs-fuse* daemon acts like a server, managing ZFS on the system through the FUSE framework. Every filesystem operation on the mounted ZFS devices from any application will be through the standard C library system calls. This results in calling the kernel's appropriate function from the virtual filesystem (VFS) interface, which will then be hooked to the FUSE module and, in turn, acts like a filesystem module through a special purpose device named */dev/fuse*. This device acts as a bridge between the ZFS implementation and *fuse* module. The *fuse* module communicates with the ZFS filesystem implementation (which in this case is *zfs-fuse*), through the FUSE library *libfuse* which has functions similar to that of VFS's interface. The user program returns results for the filesystem request in the required format through the FUSE framework to the application.

Getting started

The latest source code of ZFS on FUSE can be downloaded from the project site—www.wizy.org/wiki/ZFS_on_FUSE.

It is available in two forms, as a release version packed as a bzip file or directly in source form from the Mercurial repository. Installing from the source requires that we use *scons* instead of *make*, though the command and options are almost the same for both. It's better you read the README and INSTALL files in the source directory before proceeding. Besides, for certain distributions like Gentoo, Debian, Fedora, Ubuntu, etc, *zfs-fuse* is available via the regular package management system making the installation much easier. Please use your package manager and search for "zfs".

Installation on Fedora 10

As I was using Fedora 10 while testing ZFS, my commands and configuration files are more specific to Fedora, though with minor variations the same should apply to most distros.

First install the *zfs-fuse* package using the command [all commands from here on should be executed as the root user, unless otherwise mentioned]:

```
yum install zfs-fuse
```

This installed *zfs-fuse* version 0.5 on my system that has Fedora 10.

Setting up ZFS

Before executing any commands, it should be verified that *zfs-fuse* daemon is running.

```
pgrep zfs-fuse
```

If it's not, issue the following code:

```
service zfs-fuse start
```

...or directly run the script file as follows:

```
/etc/init.d/zfs-fuse start
```

Managing ZFS

After making sure that the *zfs-fuse* daemon is running, we need to have a ZFS pool comprising of one or more devices. We will create a pool, say 'K7', representing a group with many users, each having their own filesystems on 'K7'. A user, say 'ajc', will have his own filesystem, which will be mounted under 'K7' with the same user name along with the required properties.

```
zpool create K7 sda10
```

This will create a pool named 'K7' using the */dev/sda10* device. You can also give the full path as */dev/sda10* instead of just *sda10*. However, it's not required since *zfs-fuse* will search for any device by default in this directory. If the *-n* option is specified after *create*, then no pool will be created. This will cause just a dry-run, which ends up showing the layout of ZFS after the execution of that